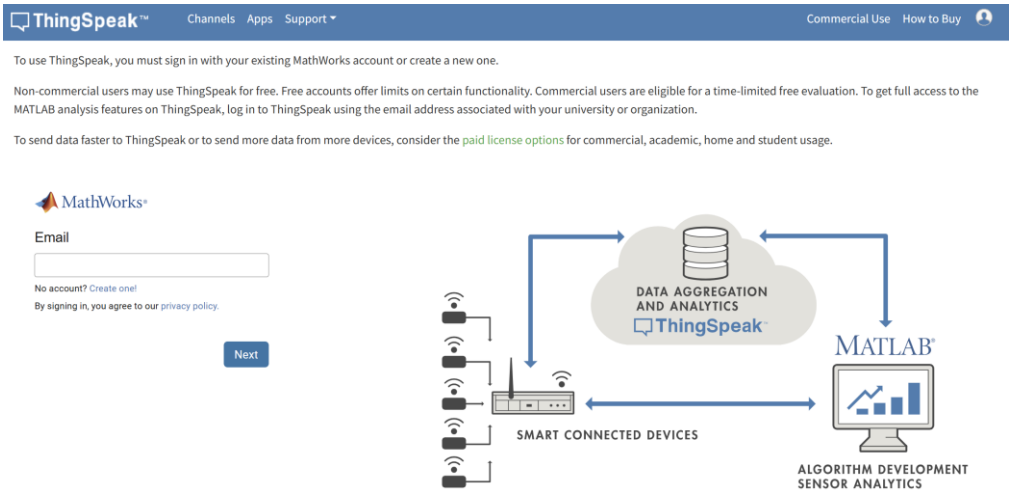
ThingSpeak and Nano ESP32

STEP 1: Creating ThingSpeak Account

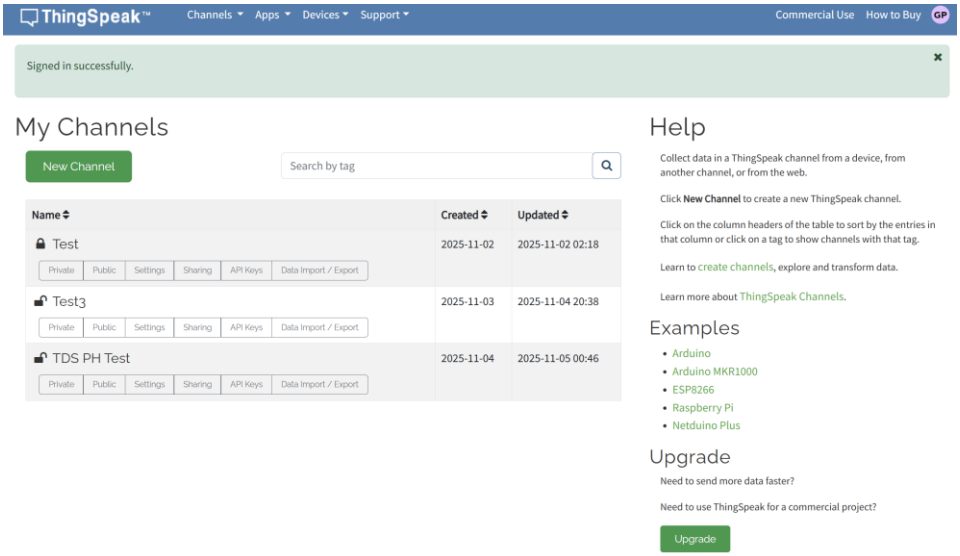
Login or Signup for ThingSpeak using your CSU Outlook Email ID using the following link:

Login: https://thingspeak.mathworks.com/login

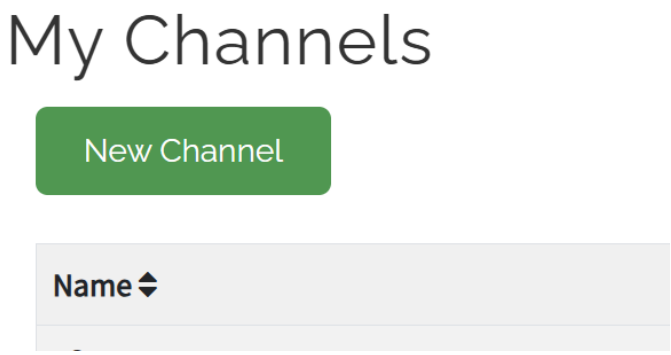


STEP 2: Creating a Channel

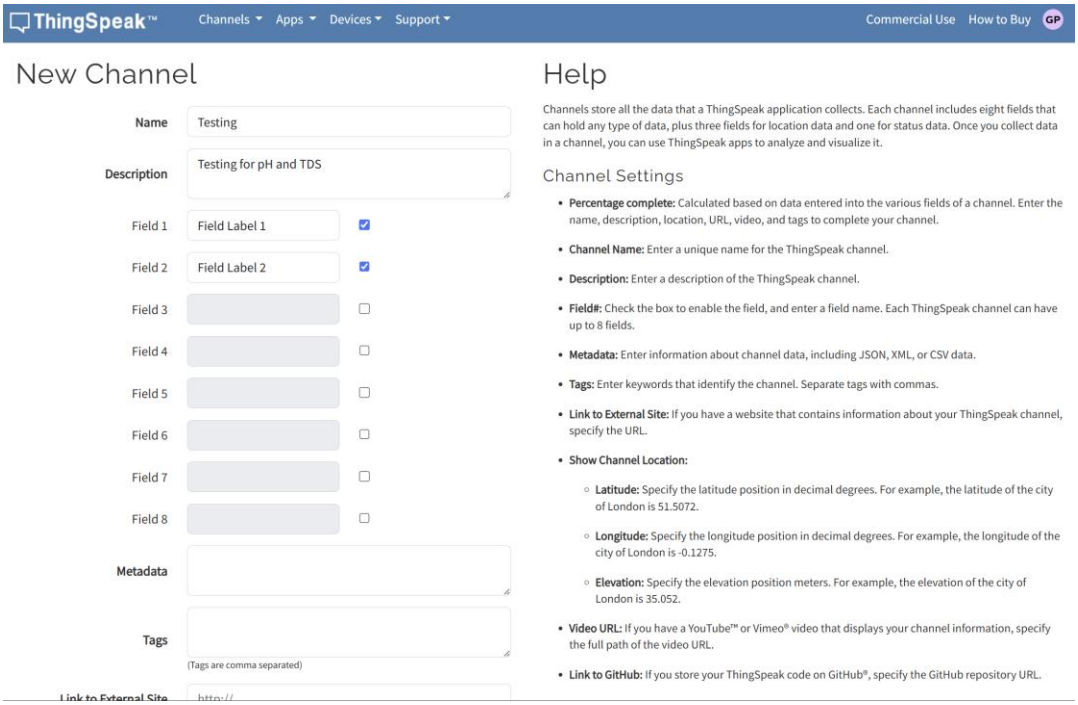
After you login, you will be take to the "**My Channels**" page



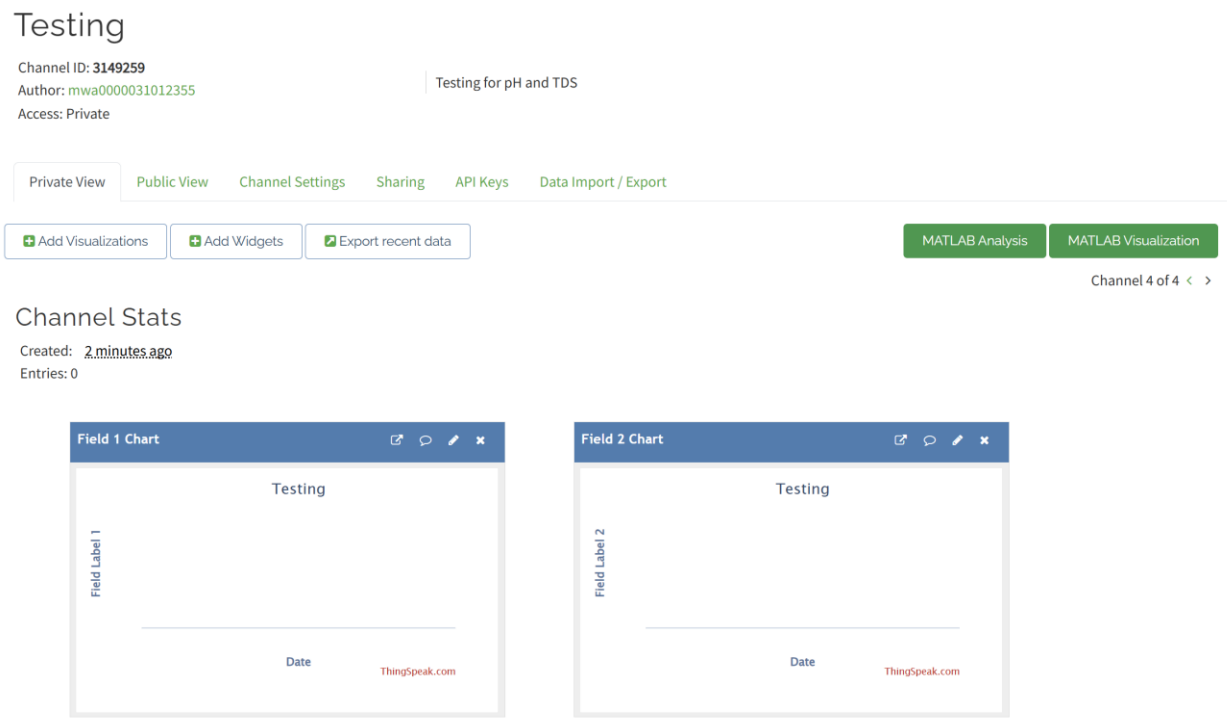
Here click on the "***New Channel***" button:



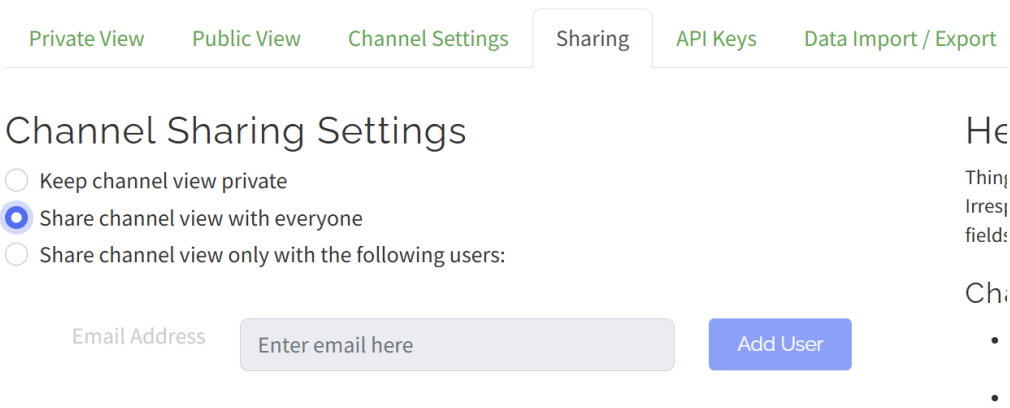
Now give the channel the desired name and description and enable as many fields as required, for our test case we only need two fields



Scroll down and hit on "**Save Channel**" which will create the channel which will look like something below:

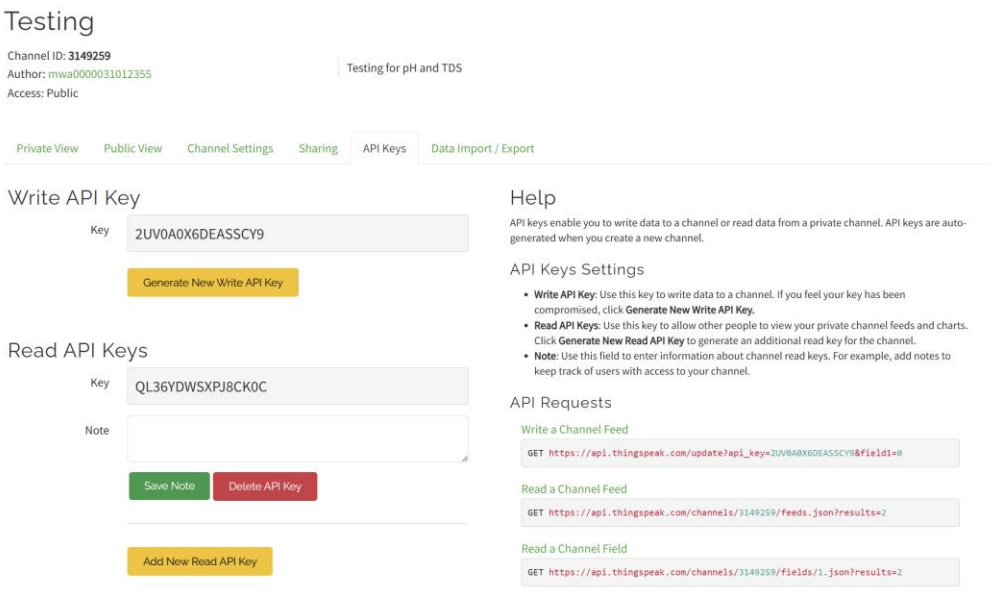


Click on sharing and change the selection to the second option:



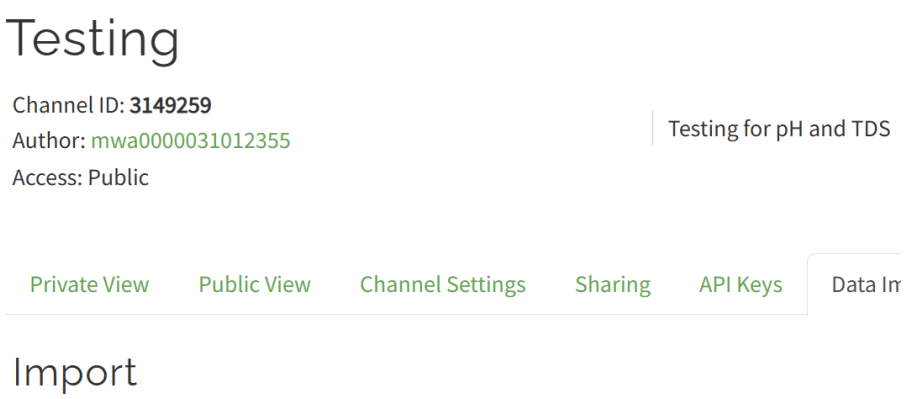
STEP 3: Noting down API key and Channel ID

Now select on API Keys and it will display something like this:



Note down the "**Write API Key**"

And from the left corner of the screen note down the channel ID:



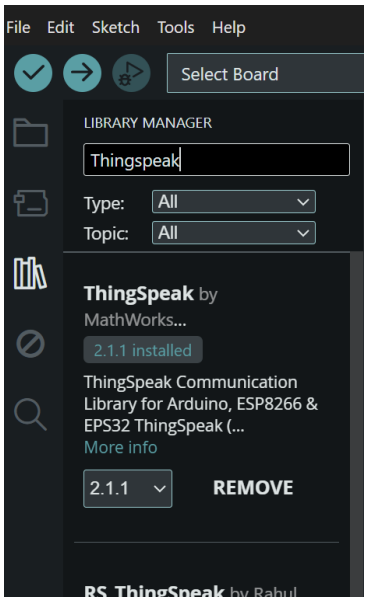
These details will be useful in later sections of this experiment.

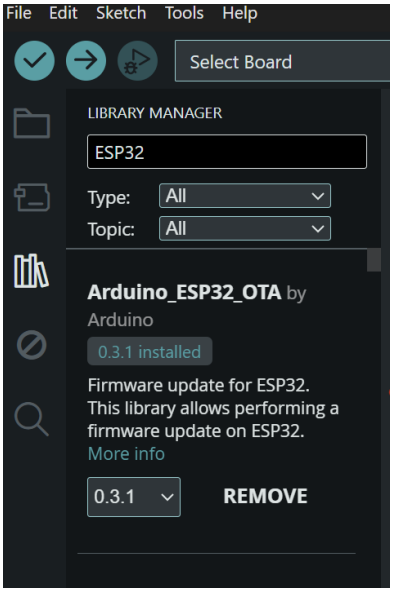
STEP 4: Arduino Nano ESP32 Code

If you have not downloaded the Arduino IDE please download it from the following link. It is free!
https://www.arduino.cc/en/software/

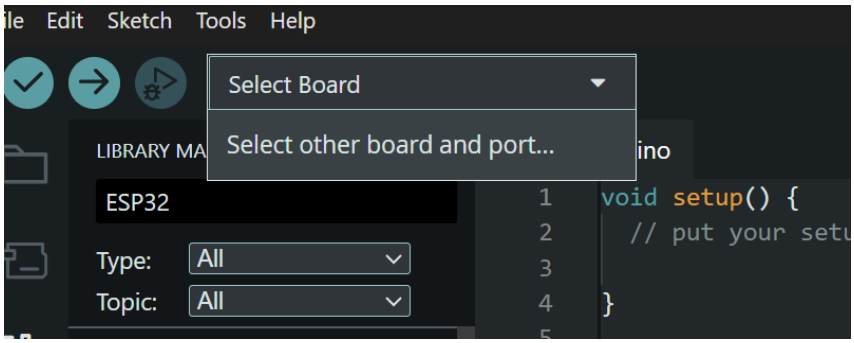
Once installed, open the application and it will look something like this:

Now on the side panel, select the third option "**Library Manager**" and search for ThingSpeak and ESP32 Libraries.

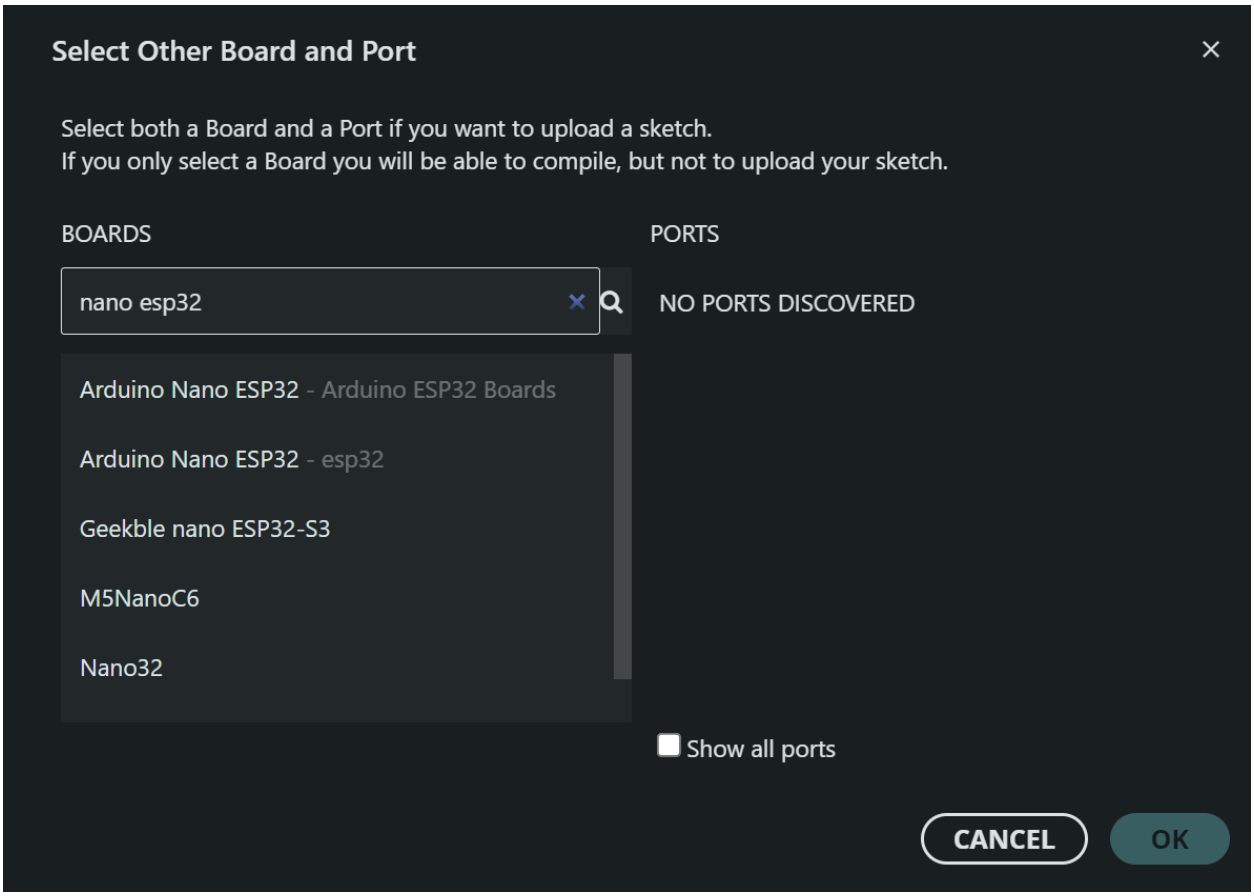




Now click on “**Select Board”** at the top of the screen just below the Menu options



It will show a dialog box, in it, search Arduino Nano ESP32, select the first option and click OK. If it is not already installed then when you select it, you will be prompted to install it, click on “Install all dependencies”.



STEP 5: Code and Execution

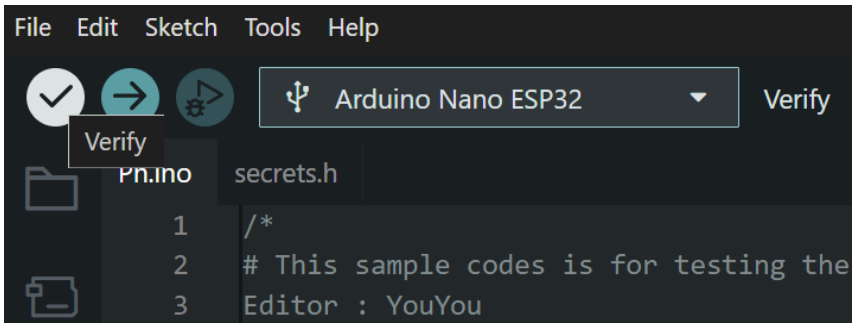
Copy paste the following code into the new sketch document and name it “**Test**”:

```
/*
# This sample codes is for testing the pH meter V1.0. #
Editor : YouYou
# Date : 2013.10.21 #
Ver : 0.1
# Product: pH meter #
SKU : SEN0161
*/
#define TdsSensorPin A2
#define VREF 3.3     // <-- FIX: Use 3.3V for ESP32/ESP8266
int analogBufferIndex = 0, copyIndex = 0;
float averageVoltage = 0, tdsValue = 0, temperature = 25;
unsigned long int avgValue2; //Store the average value of the sensor feedback void
#define Offsettds 800

#define SensorPin A0 //pH meter Analog output to Arduino Analog Input 0
#define TDSpin A1
#define Offset -5 //deviation compensate
#define TS_ENABLE_SSL // For HTTPS SSL connection

#include <WiFiClientSecure.h>
#include "secrets.h"
#include "ThingSpeak.h" // always include thingspeak header file after other header files and custom macros
unsigned long int avgValue; //Store the average value of the sensor feedback void

char ssid[] = SECRET_SSID;   // your network SSID (name)
char pass[] = SECRET_PASS;   // your network password
int keyIndex = 0;          // your network key Index number (needed only for WEP)
WiFiClientSecure  client;

unsigned long myChannelNumber = SECRET_CH_ID;
const char * myWriteAPIKey = SECRET_WRITE_APIKEY;
String myStatus = "";

void setup() {
  Serial.begin(115200);

  // Set pin modes for both sensors
  pinMode(SensorPin, INPUT);
  pinMode(TdsSensorPin, INPUT);

  Serial.println("Ready"); //Test the serial monitor

  while (!Serial) {
    ; // wait for serial port to connect. Needed for Leonardo native USB port only
  }

  WiFi.mode(WIFI_STA);
```

```
  ThingSpeak.begin(client);  // Initialize ThingSpeak
  client.setInsecure(); // Needed for ESP8266/ESP32 SSL connection
}

void loop() {
  // Connect or reconnect to WiFi
  if(WiFi.status() != WL_CONNECTED){
    Serial.print("Attempting to connect to SSID: ");
    Serial.println(SECRET_SSID);
    while(WiFi.status() != WL_CONNECTED){
      WiFi.begin(ssid, pass);  // Connect to WPA/WPA2 network.
      Serial.print(".");
      delay(5000);
    }
    Serial.println("\nConnected.");
  }

  // --- 1. pH SENSOR READING (Corrected) ---
  int buf[10];
  for(int i=0; i<10; i++){ //Get 10 sample values
    buf[i]=analogRead(SensorPin);
    delay(10);
  }
  for(int i=0; i<9; i++){ //sort the analog from small to large
    for(int j=i+1; j<10; j++){
      if(buf[i]>buf[j]){
        int temp=buf[i];
        buf[i]=buf[j];
        buf[j]=temp;
      }
    }
  }
  avgValue=0;
  for(int i=2; i<8; i++){ //take the average value of 6 center samples
    avgValue+=buf[i];
  }

  avgValue /= 6; // Get the average analog reading

  // <-- FIX: Use VREF (3.3V) and 4095.0 (for 12-bit ADC on ESP32)
  float phVoltage = (float)avgValue * VREF / 4095.0;
  float phValue = 3.5 * phVoltage + Offset; //convert the voltage into pH


  // --- 2. TDS SENSOR READING (Simplified to prevent crash) ---
  int tdsBuffer[10];
  for (int i = 0; i < 10; i++) {
    tdsBuffer[i] = analogRead(TdsSensorPin);
    delay(10);
  }

  // Get average of 10 samples
  avgValue2 = 0;
```

```
  for (int i = 0; i < 10; i++) {
    avgValue2 += tdsBuffer[i];
  }
  avgValue2 /= 10;

  // <-- FIX: Use VREF (3.3V) and 4095.0 (for 12-bit ADC on ESP32)
  averageVoltage = avgValue2 * (float)VREF / 4095.0;
  float compensationCoefficient = 1.0 + 0.02 * (temperature - 25.0);
  float compensationVolatge = averageVoltage / compensationCoefficient;
  tdsValue = (133.42 * compensationVolatge * compensationVolatge * compensationVolatge - 255.86 *
compensationVolatge * compensationVolatge + 857.39 * compensationVolatge) * 0.5;

  // Print values to Serial Monitor
  Serial.print("TDSValue:");
  Serial.print(tdsValue, 0);
  Serial.println("ppm");
  Serial.print(" pH:");
  Serial.print(phValue,2);
  Serial.println("");


  // --- 3. SEND TO THINGSPEAK ---

  ThingSpeak.setField(1, phValue);
  ThingSpeak.setField(2, tdsValue);

  // Set a status message
  if(phValue > tdsValue){
    myStatus = String("field1 is greater than field2");
  }
  else if(phValue < tdsValue){
    myStatus = String("field1 is less than field2");
  }
  else{
    myStatus = String("field1 equals field2");
  }

  ThingSpeak.setStatus(myStatus);

  // Write to the ThingSpeak channel
  int x = ThingSpeak.writeFields(myChannelNumber, myWriteAPIKey);
  if(x == 200){
    Serial.println("Channel update successful.");
  }
  else{
    Serial.println("Problem updating channel. HTTP error code " + String(x));
  }
  delay(20000); // Wait 20 seconds to update the channel again
}
```

Now click on Verify, you can find it in the top left corner of the panel



Once you are done with verification, now in a separate text document copy paste the following lines of code and save it with the name "**secrets.h**":

```
// Use this file to store all of the private credentials
// and connection details

#define SECRET_SSID " MySSID "     // replace MySSID with your Wifi network name
#define SECRET_PASS " MyPassword "  // replace MyPassword with your WiFi password

#define SECRET_CH_ID 0000000         // replace 0000000 with your channel number
#define SECRET_WRITE_APIKEY "XYZ"  // replace XYZ with your channel write API Key
```

NOTE: You need to store this file in the same folder/ directory as the "***test.ino"*** of the Arduino file. Remember, you noted down the channel ID and the Channel Write API Key.

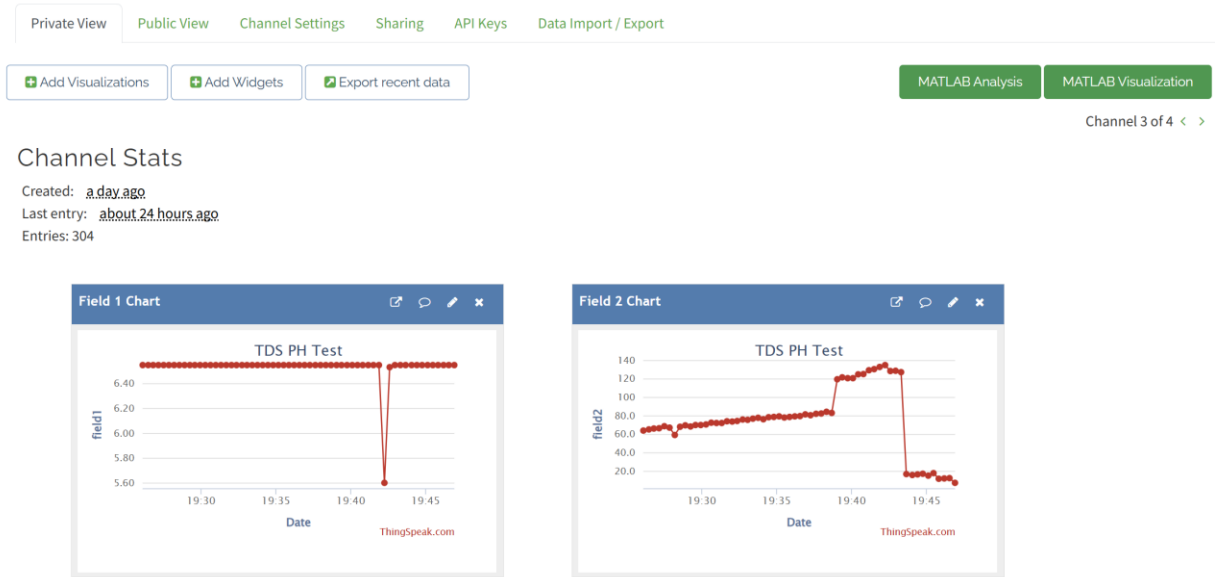
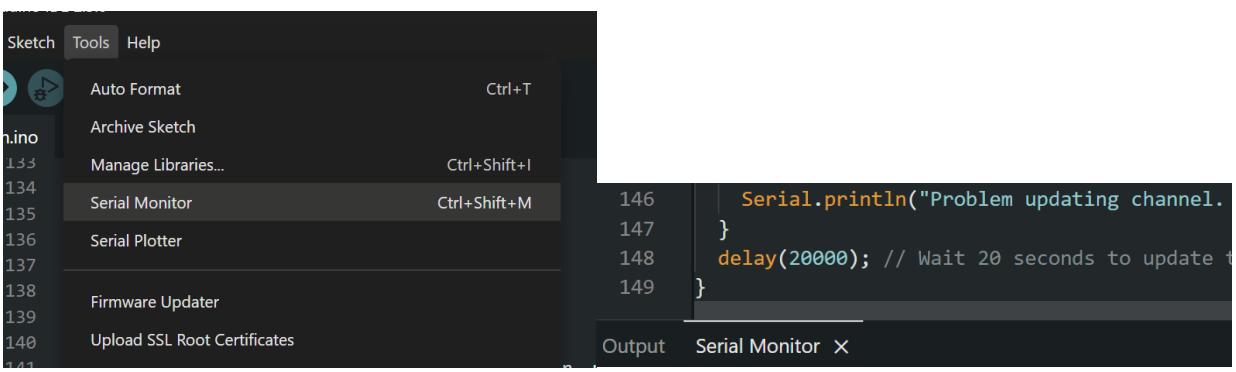
After the above two tasks, Click on upload (Right Beside Verify), which will upload ***"test.ino"*** onto the Arduino Nano ESP32.

Once that is done wire up the Arduino Board, TDS Sensor, pH Sensor and power supply based on the description below and turn on the circuit.

Arduino Nano	GND to '-ve' of power supply VBUS to '+ve' of power supply
TDS Sensor	GND to '-ve' of power supply VCC to '+ve' of power supply S (Signal) to A2 pin of Arduino Nano
pH Sensor	GND to '-ve' of power supply 2nd GND to GND of Arduino Nano V+ to '+ve' of power supply Po to A0 of Arduino Nano

STEP 6: Results

Now you can go back to your ThingSpeak account to observe the data being plotted like following:



You can observe the same values in the serial monitor of the Arduino IDE: